BusterNeece Link to events.php. 55988b6 · 3 years ago

105 lines (75 loc) · 5.84 KB

Preview Code Blame Raw

Extending AzuraCast with Plugins

This repository contains documentation for the AzuraCast plugin system, as well as a working example of some of the more common modifications that might be made to the AzuraCast system via plugins.

More information on plugins is available via the AzuraCast Documentation.

Including Plugins

Plugins are automatically discovered if they're located in the `/plugins` directory relative to the main AzuraCast installation. Plugins are ignored by the parent AzuraCast instance, so you can update your instance any time you like without worrying about your plugins being removed.

Docker Installations

You can clone the plugin directory anywhere you want on the host machine, though it's recommended to have it as a subdirectory of your AzuraCast install directory (like `/var/azuracast`). To mount the plugin as a volume so Docker recognizes it, you should create a new file named `docker-compose.override.yml` alongside the existing `docker-compose.yml` file. If you already have such a file, you can update it to include the extra lines.

```
services:
  web:
```

```
    environment:
      AZURACAST_PLUGIN_MODE: true
    volumes:
      - ./path_to_plugin:/var/azuracast/www/plugins/example-plugin
```

Make sure to restart the Docker containers afterward (using `./docker.sh restart`).

Naming Convention

Autoloaded Files

The following files are automatically loaded along with the relevant section of code:

- `/services.php`: Register or extend services with the application's Dependency Injection (DI) container.
- `/events.php`: Register listeners for the Event Dispatcher.

Classes

AzuraCast autoloads PHP files inside `/plugins/(plugin_name)/src` as long as they are in the appropriate namespace.

The function used to convert from the plugin folder name into the PHP class name is:

```php
<?php
return str_replace([' ', '_', '-'], '', ucwords($word, ' _-'));
```

For example, `/plugins/example-plugin/src` will autoload classes in the `\Plugin\ExamplePlugin` namespace.

The Event Dispatcher

Most of the extensibility of plugins comes from events that use the EventDispatcher in AzuraCast. Both classes from inside AzuraCast and plugins are registered as "listeners" to common events that are dispatched by the system, so you can override or modify the core application's responses simply by adding your own listeners in the right order.

See the `events.php` file included in this sample repository for an example of common events to listen to.

As you can see, each event listener that you register has to provide the event that it listens to as a callable to the `addListener` method's first parameter, and each event listener receives an instance of that event class complete with relevant metadata already attached. Listeners also have a priority (the last argument in the function call); this number can be positive or negative, with the default handler tending to be around zero. Higher numbers are dispatched before lower numbers.

Below is a listing of the events that can be overridden by plugins:

General Events

- `\App\Event\BuildConsoleCommands` : Register commands that will be available for invocation via the command-line interface (CLI).
- `\App\Event\BuildRoutes` : Add URL routes that will direct to controllers.
- `\App\Event\BuildView` : Configure the template engine and register your own template folders.
- `\App\Event\BuildPermissions` : Register new permissions that are used by the Access Control List (ACL)
- `\App\Event\GetNotifications` : Register new notifications shown to logged in users on the main dashboard page.
- `\App\Event\GetSyncTasks` : Register new synchronized (cron) tasks to happen in the background automatically.

Media Events

- `\App\Event\Media\GetAlbumArt` : Return the album art for a given track.
- `\App\Event\Media\ReadMetadata` : Fetch metadata about a media file.
- `\App\Event\Media\WriteMetadata` : Write metadata changes back to the media file.

NGINX Events

- `\App\Event\Nginx\WriteNginxConfiguration` : Write the per-station custom nginx configuration section.

Radio Events

- `\App\Event\Radio\AnnotateNextSong` : Convert the metadata about a track into the "annotations" format used by Liquidsoap.
- `\App\Event\Radio\BuildQueue` : Build the upcoming song playback queue for a station.

- `\App\Event\Radio\GenerateRawNowPlaying` : Ping the local mount points and remote relays to assemble a "raw" Now Playing response that AzuraCast will add rich metadata to.
- `\App\Event\Radio\WriteLiquidsoapConfiguration` : Add custom code to the Liquidsoap configuration for a given station.